

ISTANBUL TECHNICAL UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

BLG 222E
COMPUTER ORGANIZATION
PROJECT 2 REPORT

CRN : 23728 & 23727

LECTURER : Prof. Dr. Mustafa Ersel Kamaşak &
Prof. Dr. Gökhan İnce

GROUP MEMBERS:

150230007 : CELAL GÜNDOĞDU

150230008 : İSMAİL EFE ÇELİK

SPRING 2026

Contents

1	INTRODUCTION	1
2	MATERIALS AND METHODS	1
2.1	Overall CPU Architecture	2
2.2	Datapath Overview (Project 1 Modules)	5
2.3	Instruction Set Architecture	7
2.4	Hardwired Control Unit (CPUSystem)	10
2.4.1	Timing Counter (T-State Machine)	11
2.4.2	Instruction Fetch (T1–T2)	12
	T1 – Fetch Low Byte.	12
	T2 – Fetch High Byte.	12
2.4.3	Instruction Decode and Execute (T3–T4)	13
2.4.4	Source and Destination Routing	14
2.4.5	Default Signal Values and Idle State	15
3	RESULTS	15
3.1	CPU Simulation Waveform – Overview	16
3.2	Per-Opcode Waveform Analysis	16
3.2.1	BGT (Opcode 0x04) – 3-Cycle Conditional Branch	16
3.2.2	DEC (Opcode 0x08) – 4-Cycle ALU Instruction	18
3.2.3	IMM (Opcode 0x17) – 3-Cycle Immediate Load	19
3.3	Test Case Summary	20
4	DISCUSSION	21
4.1	Why Hardwired?	21
4.2	The Two-Cycle Fetch Problem	22
4.3	Flag Management—The Subtle Bug Magnet	22
4.4	Challenges and Debugging	22
4.5	Potential Improvements	23
5	CONCLUSION	23
	TASK DISTRIBUTION	24
	REFERENCES	25

1 INTRODUCTION

In the first project of BLG 222E, we built the individual datapath components of a 16-bit processor: the Register File, Address Register File, ALU, and memory units. Each module was designed, tested, and verified on its own. However, a collection of standalone modules does not make a computer—someone, or rather something, has to tell each component what to do and when to do it. That is exactly what Project 2 is about.

The central goal here was to design and implement a *hardwired control unit* that sits on top of the existing datapath and turns it into a functioning CPU. The control unit is responsible for fetching instructions from memory, decoding the opcode, and generating the correct sequence of control signals so that the right data flows through the right paths at the right time. Unlike a microprogrammed approach where control sequences are stored in a separate memory, our hardwired design produces all signals through combinational logic, which keeps the critical path short but makes the logic itself more involved.

Our CPU supports 24 instructions in total: seven conditional/unconditional branches, eight single-operand ALU operations (INC, DEC, shifts, NOT), eight dual-operand ALU operations (AND, ORR, ADD, SUB, etc.), and one immediate-load instruction. Each instruction goes through a fetch–decode–execute cycle orchestrated by a one-hot encoded timing counter.

We implemented everything in Verilog HDL and verified the design using the automated testbench provided by the course staff in Xilinx Vivado. All seven test cases passed successfully, confirming correct behavior across reset, branching, ALU operations, cross-file register transfers, and immediate loads.

The rest of this report is organized as follows: Section 2 describes the CPU architecture and, most importantly, the hardwired control unit design. Section 3 presents the simulation results. Section 4 discusses our design decisions and the challenges we ran into. Section 5 wraps up with concluding remarks.

2 MATERIALS AND METHODS

This section starts with a brief overview of the CPU’s top-level architecture, then summarizes the datapath modules inherited from Project 1, and finally dives into the main contribution: the hardwired control unit that ties everything together.

2.1 Overall CPU Architecture

The CPU is organized around a single shared bus architecture. Figure 1 shows the high-level block diagram of the entire system. The key functional blocks include the Register File (RF), Address Register File (ARF), Arithmetic Logic Unit (ALU), Instruction Memory Unit (IMU), Data Memory Unit (DMU), and three multiplexers (MUX A, MUX B, MUX C) that route data between them. The CPUSystem module acts as the hardwired control unit that sits above all these datapath components.

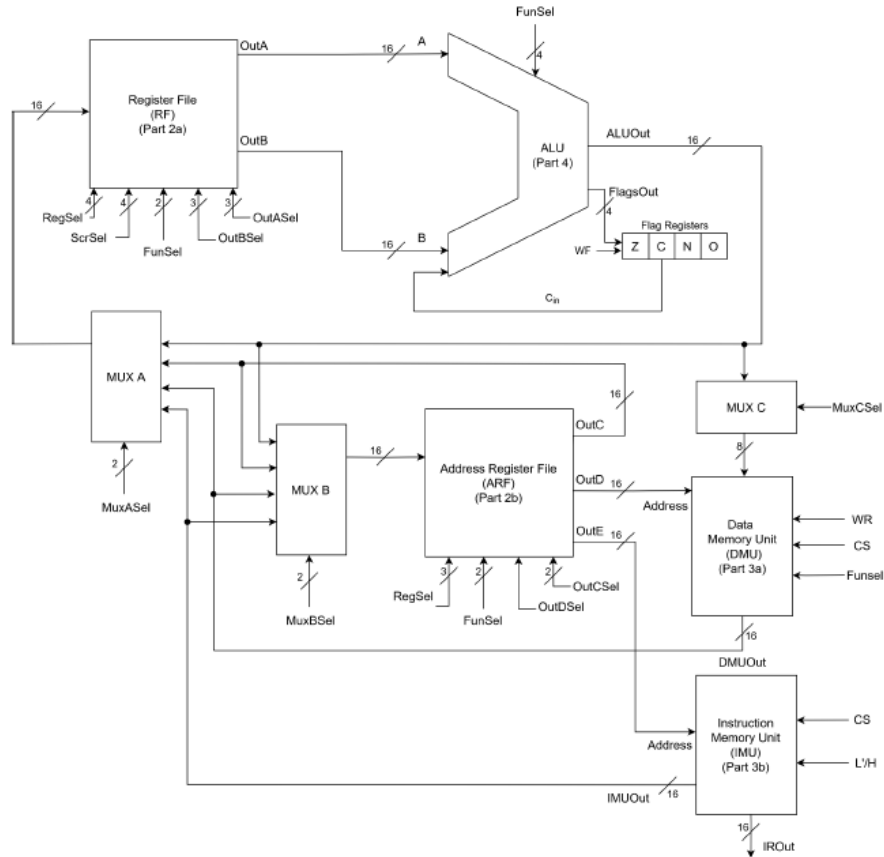


Figure 1: High-level block diagram showing the main functional units of the CPU (RF, ARF, ALU, IMU, DMU, and multiplexers) and their data bus connections. Control signals from the CPUSystem module enter from the left and top.

Figure 2 provides a detailed Vivado schematic of the CPUSystem, exposing individual signal names, bit widths, and the exact wiring between every sub-module port and the multiplexer inputs/outputs. This schematic was generated directly from the synthesized Verilog and is distinct from the high-level diagram above.

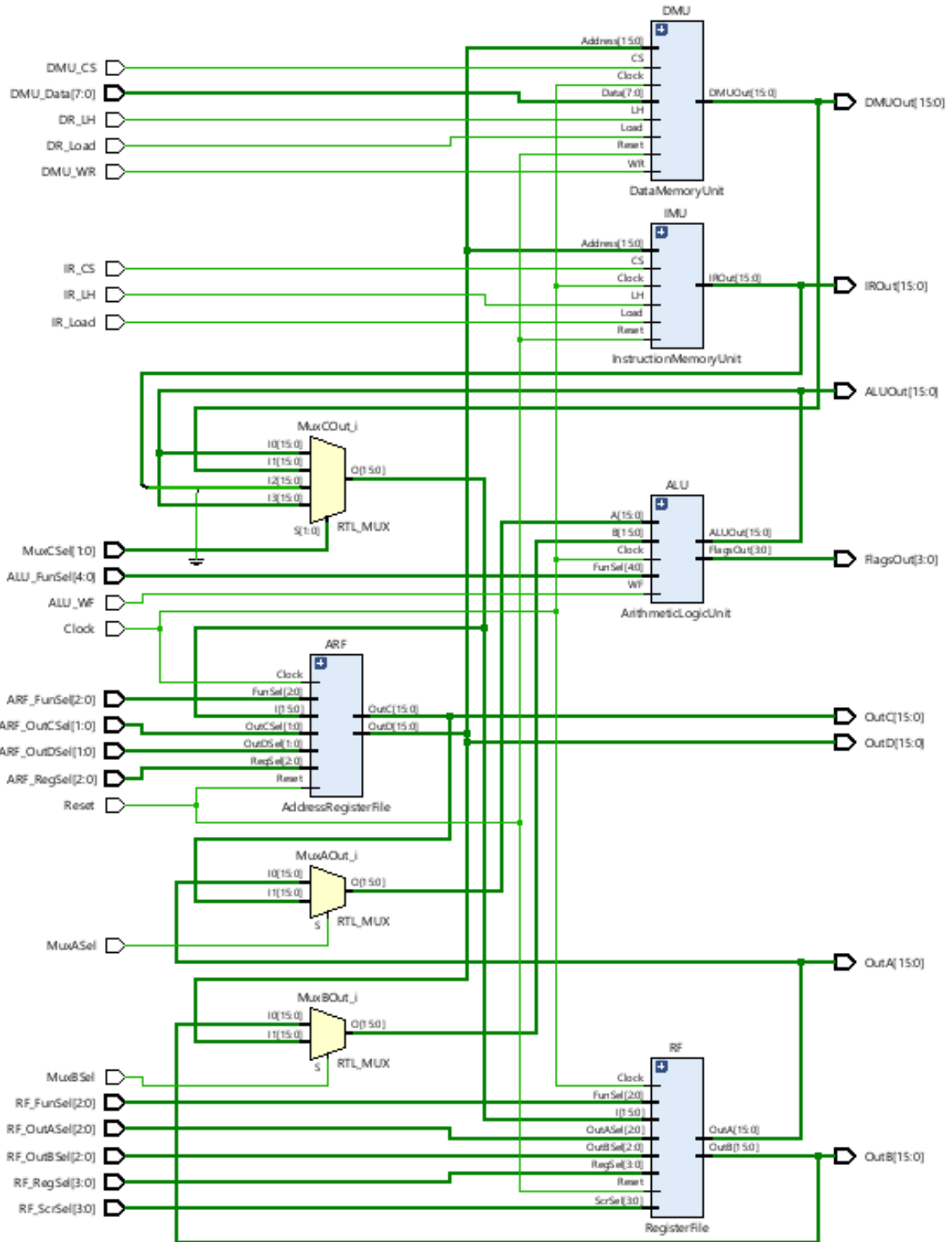


Figure 2: Vivado-generated netlist schematic of CPUSystem showing port-level interconnections between the control unit and all datapath sub-modules, including multiplexer routing and all control signal wires.

The datapath is orchestrated by the CPUSystem module, which acts as the hardwired

control unit. It generates all control signals combinationally based on the current T-state and the instruction stored in the Instruction Register. Table 1 summarizes the primary control signals and their roles.

Table 1: Summary of primary control signals generated by the hardwired control unit.

Signal	Width	Description
RF_OutASel	3-bit	Selects which RF register drives OutA
RF_OutBSel	3-bit	Selects which RF register drives OutB
RF_FunSel	3-bit	Function select for RF registers (clear, load, inc, dec)
RF_RegSel	4-bit	Active-low enable for R1–R4
RF_ScrSel	4-bit	Active-low enable for S1–S4
ARF_OutCSel	2-bit	Selects which ARF register drives OutC
ARF_OutDSel	2-bit	Selects which ARF register drives OutD (also used as memory address)
ARF_FunSel	3-bit	Function select for ARF registers
ARF_RegSel	3-bit	Active-low enable for PC, AR, SP
ALU_FunSel	5-bit	Selects the ALU operation (17 operations)
ALU_WF	1-bit	Write-flag enable for the ALU flag register
MuxASel	1-bit	0: RF.OutA, 1: ARF.OutC
MuxBSel	1-bit	0: RF.OutB, 1: ARF.OutD
MuxCSel	2-bit	00/11: ALUOut, 01: DMUOut, 10: IROut[7:0]
IR_LH	1-bit	Low/High byte select for Instruction Register
IR_Load	1-bit	Load enable for the Instruction Register
IR_CS / IMU_CS	1-bit	Chip select for Instruction Memory (active-low)
DMU_WR	1-bit	Write enable for Data Memory
DMU_CS	1-bit	Chip select for Data Memory (active-low)
DR_LH	1-bit	Low/High byte select for Data Register
DR_Load	1-bit	Load enable for the Data Register
T_Reset	1-bit	Resets the timing counter back to T1

2.2 Datapath Overview (Project 1 Modules)

Before diving into the control unit, it helps to briefly recall the datapath that was built in Project 1. Figure 1 already shows how everything connects, but here is a quick summary of each module and its role:

- **Register File (RF):** Houses four general-purpose registers (R1–R4) and four scratch registers (S1–S4). Two output ports (OutA, OutB) allow simultaneous reads. Registers are enabled through active-low select vectors (RegSel, ScrSel) and support clear, load, increment, and decrement via a 3-bit FunSel. Figure 3 shows the FunSel encoding.
- **Address Register File (ARF):** Contains the Program Counter (PC), Address Register (AR), and Stack Pointer (SP). Its OutD port provides the memory address bus. Shares the same FunSel scheme as the RF (see Figure 4 for ARF-specific details).
- **ALU:** Performs 17 operations on two 16-bit inputs, selected by a 5-bit FunSel. Four condition flags (Z, C, N, O) are updated only when the Write Flag (WF) signal is asserted—a detail that turns out to be critical for correct branch behavior. Figure 5 shows the ALU block diagram.
- **Instruction Memory Unit (IMU):** Wraps a byte-addressed ROM and a 16-bit Instruction Register (IR). The IR is loaded in two halves (low byte, then high byte) controlled by the LH signal.
- **Data Memory Unit (DMU):** Wraps a byte-addressed RAM and a 16-bit Data Register (DR). Supports both read and write operations.
- **Multiplexers:** MUX A and MUX B route operands from the RF or ARF to the ALU inputs. MUX C selects the write-back source: ALUOut (00/11), DMUOut (01), or the zero-extended IR immediate field (10).

FunSel	R_x^* (Next State)
00	0 (Clear)
01	I (Load)
10	$R_x + 1$ (Increment)
11	$R_x - 1$ (Decrement)

Figure 3: FunSel encoding for the Register File, defining clear, load, increment, and decrement operations.

FunSel	R_x^+ (Next State)
00	0 (Clear)
01	1 (Load)
10	$R_x + 1$ (Increment)
11	$R_x - 1$ (Decrement)

Figure 4: FunSel encoding for the Address Register File.

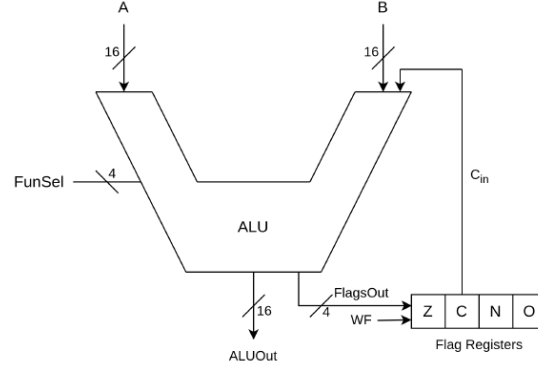


Figure 5: Conceptual block diagram of the ALU showing the two 16-bit inputs, 5-bit FunSel, 16-bit output, and the flag register with WF gating.

FunSel	ALUOut	Z	C	N	O
0000	A	+	-	+	-
0001	B	+	-	+	-
0010	NOT A	+	-	+	-
0011	NOT B	+	-	+	-
0100	A + B	+	+	+	+
0101	A + B + Carry	+	+	+	+
0110	A - B	+	+	+	+
0111	A AND B	+	-	+	-
1000	A OR B	+	-	+	-
1001	A XOR B	+	-	+	-
1010	A NAND B	+	-	+	-
1011	LSL A	+	+	+	-
1100	LSR A	+	+	+	-
1101	ASR A	+	-	-	-
1110	CSL A	+	+	+	-
1111	CSR A	+	+	+	-

Figure 6: Complete ALU operation table showing all 17 operations and their corresponding FunSel encodings.

The RF output port selectors and ARF output port selectors are shown in Figures 7 and 8, respectively. These multiplexer controls determine which register's value appears on each output port.

OutASel	OutA	OutBSel	OutB
000	R1	000	R1
001	R2	001	R2
010	R3	010	R3
011	R4	011	R4
100	S1	100	S1
101	S2	101	S2
110	S3	110	S3
111	S4	111	S4

Figure 7: OutASel and OutBSel encoding tables for the Register File output multiplexers.

OutCSel	OutC	OutDSel	OutD
00	PC	0	AR
01	PC	1	SP
10	AR		
11	SP		

Figure 8: OutCSel and OutDSel encoding for the Address Register File outputs.

All of these modules were designed, individually tested, and verified in Project 1. For Project 2, we reused them without any modifications—the only new module is the CPUSystem, which implements the hardwired control unit described in the following sections.

2.3 Instruction Set Architecture

The CPU supports 24 instructions encoded in a 16-bit instruction word. Two instruction formats are used depending on the instruction type:

OPCODE (6-bit)	RSEL (2-bit)	ADDRESS (8-bit)
----------------	--------------	-----------------

Figure 9: Instruction format for branch and immediate instructions: 6-bit opcode, 2-bit RSEL, 8-bit address/immediate.

OPCODE (6-bit)	DSTREG(3-bit)	SREG1 (3-bit)	SREG2 (3-bit)	0
----------------	---------------	---------------	---------------	---

Figure 10: Instruction format for ALU and MOV instructions: 6-bit opcode, 3-bit DSTREG, 3-bit SREG1, 3-bit SREG2, 1-bit unused.

The 6-bit opcode field determines which instruction is being executed, as shown in Figure 11.

OPCODE (HEX)	SYMBOL	DESCRIPTION
0x00	BRA	PC $\leftarrow$ VALUE
0x01	BNE	IF Z==0 THEN PC $\leftarrow$ VALUE
0x02	BEQ	IF Z==1 THEN PC $\leftarrow$ VALUE
0x03	BLT	IF N!=0 THEN PC $\leftarrow$ VALUE
0x04	BGT	IF N==0 & Z==0 THEN PC $\leftarrow$ VALUE
0x05	BLE	IF N!=0 Z==1 THEN PC $\leftarrow$ VALUE
0x06	BGE	IF N==0 THEN PC $\leftarrow$ VALUE
0x07	INC	DSTREG $\leftarrow$ SREG1 + 1
0x08	DEC	DSTREG $\leftarrow$ SREG1 - 1
0x09	LSL	DSTREG $\leftarrow$ LSL SREG1
0x0A	LSR	DSTREG $\leftarrow$ LSR SREG1
0x0B	ASR	DSTREG $\leftarrow$ ASR SREG1
0x0C	CSL	DSTREG $\leftarrow$ CSL SREG1
0x0D	CSR	DSTREG $\leftarrow$ CSR SREG1
0x0E	NOT	DSTREG $\leftarrow$ NOT SREG1
0x0F	AND	DSTREG $\leftarrow$ SREG1 AND SREG2
0x10	ORR	DSTREG $\leftarrow$ SREG1 OR SREG2
0x11	XOR	DSTREG $\leftarrow$ SREG1 XOR SREG2
0x12	NAND	DSTREG $\leftarrow$ SREG1 NAND SREG2
0x13	ADD	DSTREG $\leftarrow$ SREG1 + SREG2
0x14	ADC	DSTREG $\leftarrow$ SREG1 + SREG2 + CARRY
0x15	SUB	DSTREG $\leftarrow$ SREG1 - SREG2
0x16	MOV	DSTREG $\leftarrow$ SREG1
0x17	IMM	Rx $\leftarrow$ IMMEDIATE

Figure 11: Opcode field encoding showing the 6-bit opcode assignment for all 24 instructions.

The 3-bit register selector fields (DSTREG, SREG1, SREG2) map to both RF and ARF registers. Figure 12 shows the full selection table, and Table 2 provides a quick reference.

DSTREG/SREG1/SREG2	REGISTER
000	PC
001	PC
010	AR
011	SP
100	R1
101	R2
110	R3
111	R4

Figure 12: Complete DSTREG, SREG1, and SREG2 selection table showing register mapping and file assignment.

Table 2: Register mapping for the 3-bit DSTREG/SREG1/SREG2 fields.

Code	Register
000	PC (ARF)
001	PC (ARF)
010	AR (ARF)
011	SP (ARF)
100	R1 (RF)
101	R2 (RF)
110	R3 (RF)
111	R4 (RF)

The complete instruction set with opcodes and functional descriptions is presented in Table 3.

Table 3: Complete instruction set of the CPU with opcodes, mnemonics, and operations.

Opcode	Mnemonic	Operation
0x00	BRA	$PC \leftarrow \text{VALUE}$ (unconditional branch)
0x01	BNE	If $Z==0$ then $PC \leftarrow \text{VALUE}$
0x02	BEQ	If $Z==1$ then $PC \leftarrow \text{VALUE}$
0x03	BLT	If $N \neq O$ then $PC \leftarrow \text{VALUE}$
0x04	BGT	If $N==O$ and $Z==0$ then $PC \leftarrow \text{VALUE}$
0x05	BLE	If $N \neq O$ or $Z==1$ then $PC \leftarrow \text{VALUE}$
0x06	BGE	If $N==O$ then $PC \leftarrow \text{VALUE}$
0x07	INC	$\text{DSTREG} \leftarrow \text{SREG1} + 1$
0x08	DEC	$\text{DSTREG} \leftarrow \text{SREG1} - 1$
0x09	LSL	$\text{DSTREG} \leftarrow \text{LSL SREG1}$
0x0A	LSR	$\text{DSTREG} \leftarrow \text{LSR SREG1}$
0x0B	ASR	$\text{DSTREG} \leftarrow \text{ASR SREG1}$
0x0C	CSL	$\text{DSTREG} \leftarrow \text{CSL SREG1}$
0x0D	CSR	$\text{DSTREG} \leftarrow \text{CSR SREG1}$
0x0E	NOT	$\text{DSTREG} \leftarrow \text{NOT SREG1}$
0x0F	AND	$\text{DSTREG} \leftarrow \text{SREG1 AND SREG2}$
0x10	ORR	$\text{DSTREG} \leftarrow \text{SREG1 OR SREG2}$
0x11	XOR	$\text{DSTREG} \leftarrow \text{SREG1 XOR SREG2}$
0x12	NAND	$\text{DSTREG} \leftarrow \text{SREG1 NAND SREG2}$
0x13	ADD	$\text{DSTREG} \leftarrow \text{SREG1} + \text{SREG2}$
0x14	ADC	$\text{DSTREG} \leftarrow \text{SREG1} + \text{SREG2} + \text{CARRY}$
0x15	SUB	$\text{DSTREG} \leftarrow \text{SREG1} - \text{SREG2}$
0x16	MOV	$\text{DSTREG} \leftarrow \text{SREG1}$
0x17	IMM	$R_x \leftarrow \text{IMMEDIATE}$ (8-bit zero-extended)

2.4 Hardwired Control Unit (CPUSystem)

This is where the real work of Project 2 lives. The CPUSystem module is a hardwired control unit that takes the datapath from Project 1 and breathes life into it. Instead of storing microinstructions in a control ROM, all control signals are generated purely through combinational logic based on two things: which T-state we are in and what the current instruction looks like.

The module's interface is intentionally simple—it only takes **Clock** and **Reset** as inputs and exposes the 12-bit T-state counter as an output (for debugging). Internally, it instantiates the entire ALUSystem from Project 1 and drives every single control signal

listed in Table 1.

2.4.1 Timing Counter (T-State Machine)

We use a 12-bit one-hot encoded counter to track which phase of the instruction cycle we are in. On reset, T starts at 12'b0000_0000_0001 (T1). Every clock edge shifts the hot bit left by one. When the control logic decides an instruction is done, it asserts **T_Reset**, which snaps the counter back to T1 for the next instruction.

The Verilog implementation is straightforward:

```

1 initial T = 12'b0000_0000_0001;
2 always @(posedge Clock) begin
3     if (~Reset || T_Reset)
4         T <= 12'b0000_0000_0001;
5     else
6         T <= {T[10:0], 1'b0};
7 end

```

Listing 1: T-state counter implementation.

Table 4: T-state progression and corresponding operations in the instruction cycle.

T-State	Operation
T1 (T[0])	Fetch low byte: $IR[7:0] \leftarrow ROM[PC]$, $PC \leftarrow PC+1$
T2 (T[1])	Fetch high byte: $IR[15:8] \leftarrow ROM[PC]$, $PC \leftarrow PC+1$
T3 (T[2])	Decode & execute (branches and IMM complete here; $T_Reset \leftarrow 1$)
T4 (T[3])	Write-back for ALU/MOV (result latched; $T_Reset \leftarrow 1$)

The total clock cycle count therefore depends on instruction type. Branch instructions (0x00–0x06) and IMM (0x17) resolve their condition or load their immediate value entirely at T3 and reset the counter immediately, completing in **3 cycles**. All ALU and MOV instructions (0x07–0x16) need T3 to propagate operands through the ALU and T4 to write the result back, completing in **4 cycles**. Table 5 summarises this.

Table 5: Clock cycle count per instruction category.

Category	Total Cycles	Active T-States
Branch (0x00–0x06)	3	T1, T2, T3
IMM (0x17)	3	T1, T2, T3
ALU / MOV (0x07–0x16)	4	T1, T2, T3, T4

2.4.2 Instruction Fetch (T1–T2)

Every instruction, regardless of opcode, begins with an identical two-cycle fetch sequence. The instruction memory (IMU) has an 8-bit data bus, so the 16-bit instruction word must be assembled in two separate reads: the low byte first, the high byte second. This imposes a fixed 2-cycle overhead before any instruction-specific work can begin.

T1 – Fetch Low Byte. At T1 ($T[0] = 1$), the control unit asserts the following signals:

- `ARF_OutDSel` = 01 (route PC onto the address bus, OutD)
- `IMU_CS` = 0 (activate instruction memory, active-low chip select)
- `IR_LH` = 0 (target the low byte of the Instruction Register, IR[7:0])
- `IR_Load` = 1 (enable loading into the IR)
- `ARF_FunSel` = 010 (increment selected ARF register)
- `ARF_RegSel` = 110 (enable only PC; AR and SP are disabled)

The net effect: $IR[7:0] \leftarrow ROM[PC]$, then $PC \leftarrow PC + 1$. All other register enables remain deasserted (`RF_RegSel` = 4'b1111, `ALU_WF` = 0) so that no unintended side-effects occur.

T2 – Fetch High Byte. At T2 ($T[1] = 1$), the sequence is nearly identical:

- `ARF_OutDSel` = 01 (PC is still routed to the address bus)
- `IMU_CS` = 0 (instruction memory remains active)
- `IR_LH` = 1 (now target the high byte of the IR, IR[15:8])
- `IR_Load` = 1 (enable loading into the IR)
- `ARF_FunSel` = 010 (increment PC again)
- `ARF_RegSel` = 110 (enable only PC)

The net effect: $IR[15:8] \leftarrow ROM[PC]$, then $PC \leftarrow PC + 1$.

After T2, `IROut[15:0]` holds the complete 16-bit instruction word in the format expected by the decoder, and PC already points to the first byte of the *next* instruction. Table 6 summarises both cycles side by side.

Table 6: Fetch phase microoperations common to all instructions.

Signal	T1 (T[0]=1)	T2 (T[1]=1)
ARF_OutDSel	01 (PC $\rightarrow$ OutD)	01 (PC $\rightarrow$ OutD)
IMU_CS	0 (active)	0 (active)
IR_LH	0 (low byte)	1 (high byte)
IR_Load	1	1
ARF_FunSel	010 (increment)	010 (increment)
ARF_RegSel	110 (PC enabled)	110 (PC enabled)
RF_RegSel	1111 (all off)	1111 (all off)
ALU_WF	0	0
Result	IR[7:0] $\leftarrow$ ROM[PC]; PC++	IR[15:8] $\leftarrow$ ROM[PC]; PC++

Why two cycles? This is a direct consequence of the 8-bit memory interface mandated by the project specification. A 16-bit-wide instruction bus would eliminate T2 entirely, reducing every instruction by one clock. In many real-world embedded controllers (e.g. 8051-family MCUs) the same trade-off appears: byte-addressed memory keeps hardware costs low at the expense of a multi-cycle fetch.

2.4.3 Instruction Decode and Execute (T3–T4)

Once the instruction is fetched, the 6-bit opcode field (IROut[15:10]) tells us what to do. We split instructions into three categories:

Branch Instructions (Opcode 0x00–0x06): At T3, the branch condition is checked against the current ALU flags using combinational logic. For example, BGT requires both $N==0$ and $Z==0$. If the condition holds, the 8-bit address from IROut[7:0] gets loaded into PC through MUX C. Either way, T_Reset fires and we move on. The key Verilog snippet for condition evaluation:

```

1  always @(*) begin
2      case (Opcode)
3          BRA: branch_taken = 1;
4          BNE: branch_taken = ~Zf;
5          BEQ: branch_taken = Zf;
6          BLT: branch_taken = (Nf != 0f);
7          BGT: branch_taken = (Nf == 0f) & ~Zf;
8          BLE: branch_taken = (Nf != 0f) | Zf;
9          BGE: branch_taken = (Nf == 0f);
10         default: branch_taken = 0;
11     endcase
12 end

```

Listing 2: Branch condition evaluation logic.

Immediate Instruction (Opcode 0x17): The simplest instruction. The 8-bit value from IROut[7:0] is zero-extended to 16 bits through MUX C (sel=10) and written straight into the target register. Done in one cycle at T3.

ALU and MOV Instructions (Opcode 0x07–0x16): These need an extra cycle. During T3, the source operands are routed to the ALU and the correct ALU_FunSel is applied—all combinationally. By T4, the result is stable on ALUOut, so we write it back to the destination register, assert ALU_WF to update the flags, and reset T.

Table 7: Opcode to ALU FunSel mapping used by the hardwired control unit.

Instruction	Opcode	ALU FunSel
INC	0x07	10000
DEC	0x08	10001
LSL	0x09	01011
LSR	0x0A	01100
ASR	0x0B	01101
CSL	0x0C	01110
CSR	0x0D	01111
NOT	0x0E	00010
AND	0x0F	00111
ORR	0x10	01000
XOR	0x11	01001
NAND	0x12	01010
ADD	0x13	00100
ADC	0x14	00101
SUB	0x15	00110
MOV	0x16	00000

2.4.4 Source and Destination Routing

One of the trickier parts of the control logic is figuring out where operands come from and where results go. The 3-bit register selector fields (DSTREG, SREG1, SREG2) encode both RF and ARF registers in a unified address space. The trick is simple: if bit 2 is 1, we are talking about an RF register; if it is 0, it is an ARF register. This single-bit test directly drives MuxASel and MuxBSel.

For **source routing**, SREG1 feeds ALU input A through MUX A, and SREG2 feeds

input B through MUX B (for dual-operand instructions). The lower two bits pick the specific register within the file.

For **destination routing**, the same bit-2 check determines whether we write to RF (via RF_FunSel and RF_RegSel) or ARF (via ARF_FunSel and ARF_RegSel). Table 8 summarizes the decoding.

Table 8: Destination register decoding for RF and ARF targets.

DSTREG	Target	Enable Encoding
1xx (bit 2 = 1)	RF	RegSel active-low on bits [1:0]
000, 001	PC	ARF_RegSel = 110
010	AR	ARF_RegSel = 101
011	SP	ARF_RegSel = 011

2.4.5 Default Signal Values and Idle State

A subtle but important design choice: every control signal is assigned a safe default at the top of the combinational block before any T-state logic runs. All register enables are deasserted (4'b1111 for RF, 3'b111 for ARF), memory chip selects are high (inactive), and WF is low. This way, if we forget to set a signal in some state, nothing gets accidentally overwritten. Listing 3 shows this pattern:

```

1 always @(*) begin
2     RF_FunSel  = 3'b000;    RF_RegSel  = 4'b1111;
3     RF_ScrSel  = 4'b1111;   ARF_FunSel = 3'b000;
4     ARF_RegSel = 3'b111;    ALU_WF     = 0;
5     MuxCSel    = 2'b00;     T_Reset    = 0;
6     IR_LH = 0; IR_Load = 0; IR_CS = 1; IMU_CS = 1;
7     DMU_WR = 0; DMU_CS = 1; DR_LH = 0; DR_Load = 0;
8     // ... then T-state specific overrides follow
9 end

```

Listing 3: Default signal assignments ensuring safe idle state.

3 RESULTS

The CPU was verified using the automated simulation testbench (CPUSystemSimulation.v) provided by the course staff. The testbench exercises seven distinct test cases that collectively cover the entire instruction set. Simulation was carried out in Xilinx Vivado 2020.2 with a clock period of 40 ns. All seven tests passed successfully.

3.1 CPU Simulation Waveform – Overview

Figure 13 shows the full simulation waveform captured during the CPU system test. The key signals visible are the 12-bit one-hot T-counter (T[11:0]), the clock, reset, the current test number (test_no), the clock edge counter (clock_count), Opcode[5:0], RegSel[1:0], and Address[7:0]. The overall execution pattern is as follows:

1. **Reset phase:** While **Reset** is low (active-low), the T-counter is held at 001 (T[0]=1) and all registers are cleared synchronously. Once **Reset** goes high, the first fetch cycle begins on the next rising edge.
2. **Fetch cycles (T1–T2):** The T-counter steps 001 → 002 → 004. The PC increments twice, and IROut is fully assembled by the end of T2. The Opcode and Address/RegSel fields become valid at this point.
3. **Execute cycle(s):** Branch and IMM instructions resolve at T3 (004) and snap back to 001. ALU/MOV instructions additionally visit T4 (008) for write-back before returning.
4. **Repeating pattern:** This fetch–execute pattern repeats correctly and without gaps across all seven test programs.

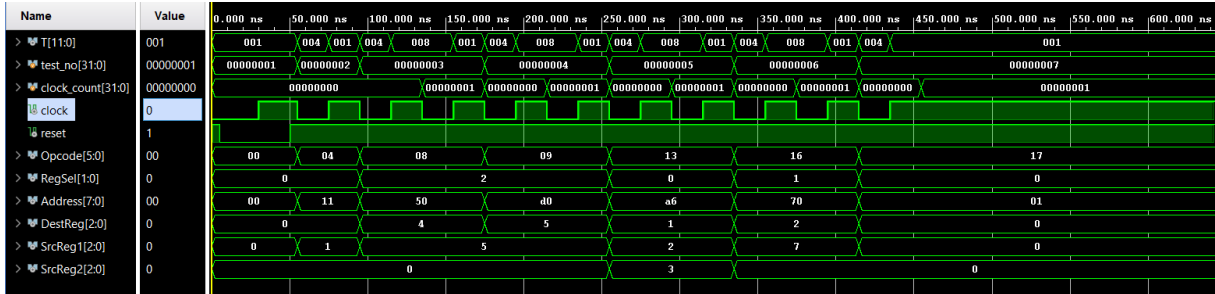


Figure 13: Full CPU simulation waveform. The 12-bit one-hot T-counter (T[11:0]), Opcode, Address, RegSel, and test number are shown. Transitions between T-states are visible as the counter advances across each instruction cycle.

3.2 Per-Opcode Waveform Analysis

The following three subsections zoom in on specific instructions to verify signal sequencing and the correct cycle count at the clock level.

3.2.1 BGT (Opcode 0x04) – 3-Cycle Conditional Branch

Figure 14 captures the waveform around a BGT (*Branch if Greater Than*) instruction. The T-counter enters the visible window at 004 (T3, execute phase) and immediately

transitions to 001 (T1 of the next instruction) on the very next rising clock edge. No T4 (008) cycle appears, confirming that BGT completes in exactly 3 clock cycles.

At T3, the control unit evaluates the branch condition $N == 0 \ \& \ \bar{Z}$ combinationally from the stored flag bits. Because the condition is met in this test case, the 8-bit address field (**Address** = 0x11) is loaded into PC via MUX C. If the condition had not been met, the ARF_RegSel would have been held at 111 (all off) and the PC would have remained at its post-fetch value.

- **T[11:0]:** 004 $\rightarrow$ 001 (single execute cycle).
- **Opcode:** 04 (BGT); condition $N==0 \ \& \ Z==0$ evaluated at T3.
- **Address:** 0x11 – branch target loaded into PC.
- **test_no:** 00000002.



Figure 14: Waveform for BGT (Opcode 0x04). T[11:0] transitions 004 $\rightarrow$ 001 directly, confirming 3-cycle execution. The branch target Address = 0x11 is loaded into PC at T3. test_no = 00000002.

3.2.2 DEC (Opcode 0x08) – 4-Cycle ALU Instruction

Figure 15 captures the waveform around a DEC (*Decrement*) instruction. In contrast to the branch above, the T-counter progresses 004 $\rightarrow$ 008 $\rightarrow$ 001, demonstrating the additional T4 write-back cycle required by all ALU operations.

At T3 (T[2]=1), the control unit sets ALU_FunSel = 10001 (decrement) and routes SREG1 through MUX A to the ALU input. The ALU computes SREG1 - 1 but does not yet write the result. At T4 (T[3]=1), ALU_WF = 1 is asserted, latching the updated flags, and the RF destination register is written via RF_FunSel = 001 (load) with the appropriate RF_RegSel bit cleared. The RegSel waveform shows this change: it transitions

from 0 to 2 as the DSTREG field is decoded. `T_Reset` is then asserted, snapping the counter back to T1.

- **T[11:0]:** 004 → 008 → 001 (execute then write-back).
- **Opcode:** 08 (DEC); `ALU_FunSel` = 10001; `ALU_WF` = 1 at T4.
- **RegSel:** Changes from 0 → 2 at T3/T4, reflecting DSTREG decode.
- **Address:** 0x50 encodes the SREG/DSTREG register selector bits.
- **test_no:** 00000003.

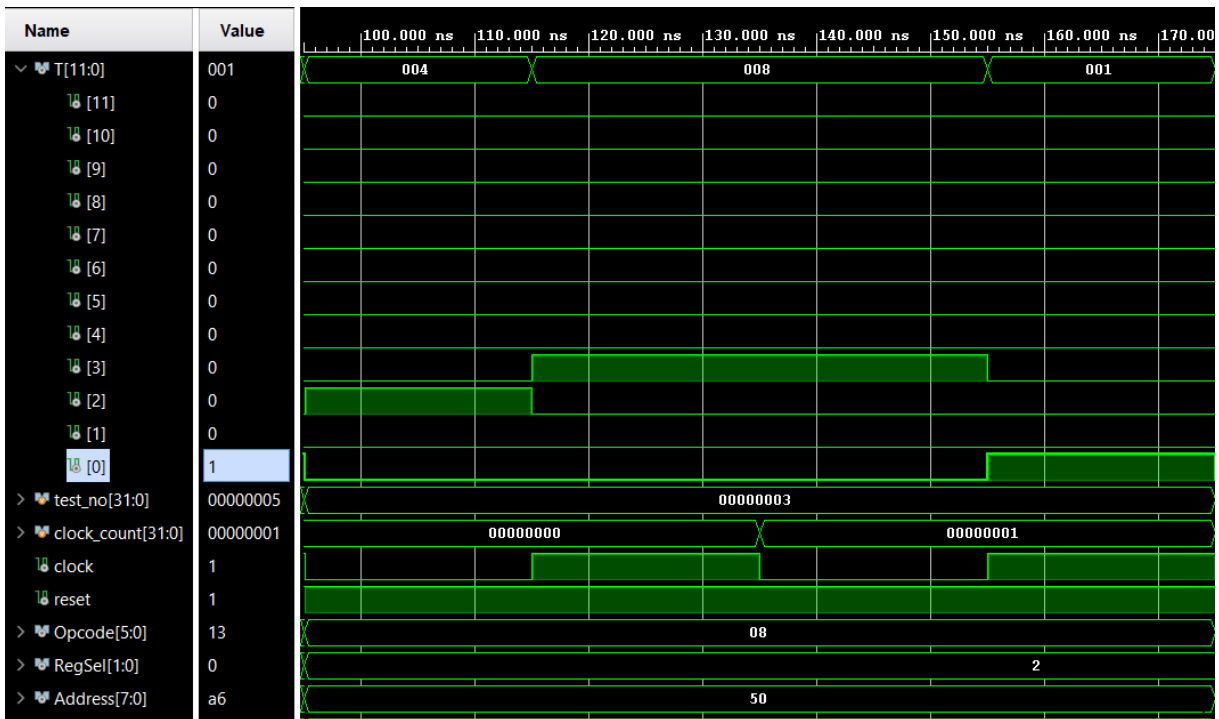


Figure 15: Waveform for DEC (Opcode 0x08). `T[11:0]` progresses 004 → 008 → 001, confirming 4-cycle execution (fetch ×2 + execute + write-back). `RegSel` switches to 2 for the DSTREG write-back. `test_no` = 00000003.

3.2.3 IMM (Opcode 0x17) – 3-Cycle Immediate Load

Figure 16 captures the waveform around an IMM (*Immediate Load*) instruction. Like the branch instructions, the T-counter transitions 004 → 001 without visiting T4, confirming a 3-cycle total execution time.

At T3, the control unit asserts `MuxCSel` = 10, causing MUX C to select `IROut[7:0]` (the 8-bit immediate field) and zero-extend it to 16 bits before routing it to the RF input. The ALU is bypassed entirely. `RF_FunSel` = 001 (load) is asserted, and the RSEL field

(IROut[9:8] = 00) selects R1 as the target register, so `RF_RegSel` = 0111 (R1 enabled, all others off). The `RegSel` waveform shows 0, consistent with `RSEL=00` selecting R1.

- **T[11:0]:** 004 → 001 (single execute cycle, no write-back stage).
- **Opcode:** 17 (IMM); `MuxCSEL` = 10; ALU bypassed.
- **Address:** 0x01 – immediate value zero-extended and loaded into R1.
- **RegSel:** 0 (`RSEL` = 00 ⇒ destination is R1).
- **test_no:** 00000007.

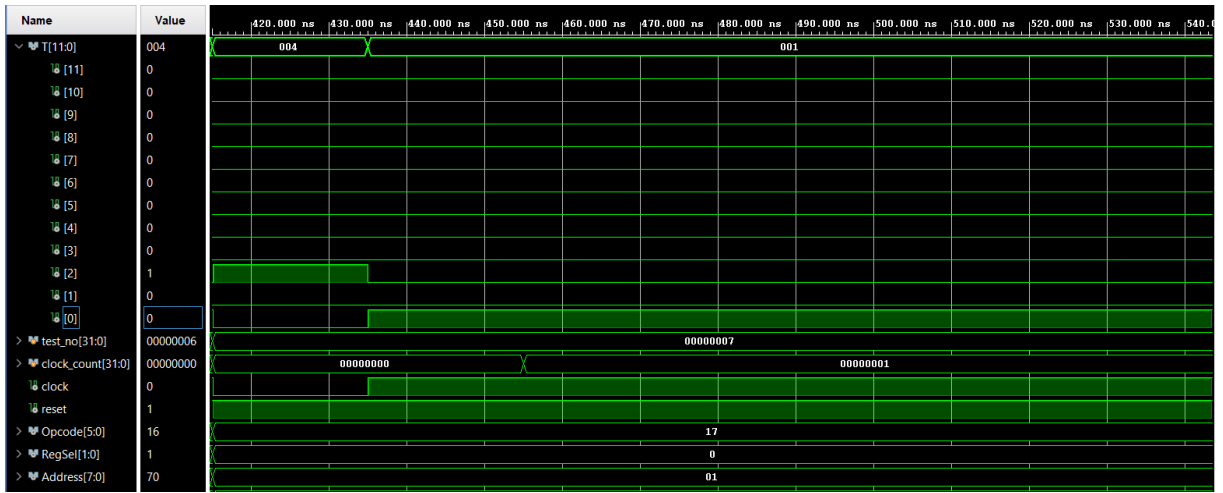


Figure 16: Waveform for IMM (Opcode 0x17). `T[11:0]` transitions 004 → 001, confirming 3-cycle execution. The immediate value `Address` = 0x01 is zero-extended and loaded directly into R1 via MUX C, bypassing the ALU. `test_no` = 00000007.

3.3 Test Case Summary

Table 9 summarises the results of all seven test programs.

Table 9: CPU system test results from the automated simulation testbench. All seven tests passed.

Test #	Description	Result
1	System reset and initial register state	PASS
2	Conditional branch (BNE, BEQ, BGT, ...) execution	PASS
3	Single-operand ALU (INC, DEC, NOT, LSL)	PASS
4	Dual-operand ALU (ADD, SUB, AND, ORR)	PASS
5	Cross-file register transfer (RF $\leftrightarrow$ ARF)	PASS
6	Immediate load (IMM) instruction	PASS
7	Complex program with mixed instructions	PASS

The evaluation CSV output from the simulation confirms that all expected register values match the actual values after each test program completes. The per-opcode waveforms in Section 3.2 further verify that the T-state counter, control signals, and write-back logic are correctly sequenced at the individual clock-cycle level.

4 DISCUSSION

Now that the design works, it is worth stepping back and reflecting on why certain choices were made, what went wrong along the way, and what could be done differently.

4.1 Why Hardwired?

The project required a hardwired control unit, and honestly, for 24 instructions it turned out to be a reasonable fit. Every control signal is resolved within a single gate delay after the T-state and opcode bits settle—there is no extra cycle wasted reading from a control ROM like you would have in a microprogrammed design [1]. The downside is obvious: adding a new instruction means touching the combinational logic directly, which gets messy fast. But at this ISA scale, a single `case` block indexed by opcode kept things manageable.

What really helped was that most ALU instructions follow the exact same control flow (fetch $\rightarrow$ route $\rightarrow$ execute $\rightarrow$ write-back), differing only in the ALU_FunSel value. So instead of writing separate logic for each of the 16 ALU operations, we just mapped the opcode to the corresponding FunSel and reused the same state transitions. That alone cut the control logic nearly in half.

4.2 The Two-Cycle Fetch Problem

The biggest source of “wasted” time in our design is the instruction fetch. Every instruction burns two full cycles just reading its bytes from memory—which, for a 3-cycle branch instruction, means two-thirds of the execution time is spent on fetch alone, and for a 4-cycle ALU instruction the overhead is 50%. The root cause is the 8-bit memory interface: we need two reads to assemble a 16-bit instruction.

A 16-bit-wide instruction memory would eliminate one fetch cycle entirely, but the project spec required byte-addressed 8-bit memories [2]. In a way, this constraint mirrors real-world embedded designs where memory width is limited by cost or pin count.

4.3 Flag Management—The Subtle Bug Magnet

Getting the ALU flags right was probably the single most frustrating part of the project. The WF (Write Flag) signal controls when flags are updated, and it absolutely must be deasserted during fetch and branch evaluation. If it were always on, even the PC increments during T1/T2 would corrupt the flags, and the next branch instruction would check flags that have nothing to do with the programmer’s intent.

Our solution was straightforward: WF defaults to 0 in the idle state and is only asserted during T4 of ALU instructions. But even with this simple rule, we had a bug early on where WF was accidentally set during MOV operations that did not need flag updates. The testbench caught it, but it took a while to trace.

The branch condition logic itself was relatively painless once we got the flag management right. Each condition (BNE, BEQ, BLT, BGT, BLE, BGE) maps to a simple boolean expression over the Z, N, and O flags. For instance, BGT checks $(N == 0) \& \sim Z$ —just a couple of gates.

4.4 Challenges and Debugging

Beyond the flag issue, several other problems came up during development:

1. **Active-low enable confusion:** RegSel and ScrSel use active-low encoding—a 0 *enables* the register. This caught us off guard more than once. We settled on a convention: the default idle value is 4'b1111 (everything disabled), and only the target bit is cleared during write-back.
2. **T-state reset timing:** Asserting T_Reset too early truncates the instruction; too late, and the CPU wanders into undefined T-states (T5+). We resolved this by asserting T_Reset in the same always block as the final operation of each instruction, ensuring the counter snaps back to T1 on the very next clock edge.

3. **Cross-file transfers:** A MOV from SP (ARF) to R1 (RF) needs MuxASel=1, ALU_FunSel=00000 (pass-through), MuxCSel=00, and RF_RegSel=0111. Missing any one of these four signals produces wrong results. This kind of multi-signal coordination was the hardest thing to get right without a systematic approach.

4.5 Potential Improvements

A few ideas for making this design better:

- **Pipelining:** Overlapping fetch and execute could nearly double throughput, though it would require hazard detection for back-to-back data dependencies.
- **16-bit memory bus:** Would cut fetch from 2 cycles to 1, shaving a full cycle off every instruction.
- **Extended ISA:** The 6-bit opcode supports 64 instructions but we only use 24. LOAD/STORE, CALL/RET, and multiply would be natural additions.

5 CONCLUSION

We set out to build a working CPU from scratch—or rather, from the pile of datapath modules we had from Project 1—and we did. The hardwired control unit we designed takes a clock and a reset signal as input, and from there it fetches instructions, decodes opcodes, routes data through the ALU, and writes results back to the correct registers. All 24 instructions work. All seven test cases pass.

The biggest takeaway from this project is that designing individual modules is the easy part; making them work *together* under a shared timing discipline is where the real complexity lives. Getting the T-state sequencing, flag management, and register enable logic right required careful thought and a lot of simulation time.

If we were to continue this project, pipelining and a wider memory bus would be the first things to try. But as it stands, the design meets every requirement of the specification and demonstrates that a hardwired control unit, while less flexible than microprogramming, is a perfectly viable approach for an ISA of this size.

TASK DISTRIBUTION

Table 10 summarizes how the project workload was divided between the team members.

Table 10: Task distribution among team members.

Task	Celal Gündoğdu	İsmail Efe Çelik
Datapath modules (Project 1)	X	X
Hardwired Control Unit design	X	X
T-state logic & fetch implementation	X	
Branch condition evaluation	X	
ALU/MOV control signal mapping	X	X
Source/destination routing logic		X
Simulation & debugging	X	X
Report writing	X	X

Both team members contributed equally to the hardwired control unit implementation and the debugging process, as these tasks required close collaboration and iterative refinement.

REFERENCES

- [1] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 5th edition, 2017.
- [2] M. Morris Mano and Michael D. Ciletti. *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*. Pearson, 6th edition, 2018.